1.Implementation of Candidate-Elimination Algorithm

Easy (Child-friendly algorithm)

Imagine you are a detective trying to figure out a secret rule (the "hypothesis") by looking at examples.

1. **Start with two guesses: One is the most specific guess (it only allows one exact thing), and the other is the most general guess (it allows everything, shown as "?").**
2. **Look at "Yes" examples: If a "Yes" example doesn't fit your specific guess, make that guess broader so it includes the new example.**
3. **Look at "No" examples: If a "No" example fits your general guess, make that general guess more specific so it correctly excludes that "No" case.**
4. **Finish: After seeing all examples, you'll have a range of rules that perfectly fit all the "Yes" examples and none of the "No" examples.**

Code

```
import pandas as pd
import numpy as np

# Load dataset
data = pd.read_csv('dataset.csv')
concepts = np.array(data.iloc[:, 0:-1])
target = np.array(data.iloc[:, -1])

def candidate_elimination(concepts, target):
    # Initialize specific hypothesis with the first positive example
    specific_h = concepts[0].copy()
    # Initialize general hypothesis with '?' (most general)
    general_h = [["?" for _ in range(len(specific_h))] for _ in range(len(specific_h))]

    for i, h in enumerate(concepts):
        if target[i] == "Yes":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    specific_h[x] = '?'
                    general_h[x][x] = '?'

        elif target[i] == "No":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    general_h[x][x] = specific_h[x]
                else:
                    general_h[x][x] = '?'

    # Filter out empty entries from general hypothesis
    general_h = [g for g in general_h if g != ['?'] * len(specific_h)]
    return specific_h, general_h
```

```
s_final, g_final = candidate_elimination(concepts, target)
print(f"Final Specific_h:\n{s_final}")
print(f"Final General_h:\n{g_final}")
```

Sample Output

```
Final Specific_h:
['sunny' 'warm' '?' 'strong' '?' '?']

Final General_h:
[['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?']]
```

2.Implementation of Decision Tree in ID3 Algorithm

Easy (Child-friendly algorithm)

Imagine you are playing a game of "20 Questions" to guess an animal.

1. **Ask the best question first: You look for the question that splits the group the most (like "Does it have wings?" instead of "Is it named Bob?").**
2. **Make groups: Based on the answer, you put the animals into different piles.**
3. **Keep asking: For each pile, you ask another great question until you have only one kind of animal left in each group.**
4. **Finish: Now you have a map of questions (a tree) that leads you to the right answer every time!**

Code

This concise version uses scikit-learn to efficiently build the ID3 tree (using Entropy) and classify a new sample.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.preprocessing import LabelEncoder

# 1. Load the dataset (Ensure Tennisdata.csv is in the folder)
data = pd.read_csv('Tennisdata.csv')

# 2. Convert text categories into numbers
le = LabelEncoder()
for col in data.columns:
    data[col] = le.fit_transform(data[col])

# 3. Separate features (X) and the answer we want to predict (y)
X = data.drop(columns=['Answer'])
y = data['Answer']

# 4. Build the Decision Tree using ID3 logic (Entropy)
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X, y)

# 5. Show the tree and predict for a new sample
print("Decision Tree Structure:\n", export_text(clf, feature_names=list(X.columns)))

# Example: Outlook=Sunny(2), Temp=Mild(2), Humidity=High(0), Wind=Strong(0)
new_sample = [[2, 2, 0, 0]]
prediction = clf.predict(new_sample)
print(f"Prediction for new sample: {prediction}")
```

Sample Output

Decision Tree Structure:

```
 |--- Outlook <= 0.50
|   |--- value: [1.00]
|--- Outlook >  0.50
|   |--- Humidity <= 0.50
|   |   |--- value: [0.00]
|   |--- Humidity >  0.50
|   |   |--- Wind <= 0.50
|   |   |   |--- value: [1.00]
```

Prediction for new sample: [0]

3.Artificial Neural Network with Backpropagation

Easy (Child-friendly algorithm)

Imagine you are teaching a robot to recognize pictures of cats and dogs.

1. **First Guess: The robot makes a random guess for each picture.**
2. **Check for Mistakes: You tell the robot if it was right or wrong. If it was wrong, you show it exactly how far off it was (this is the "error").**
3. **Fixing the Brain: The robot "thinks backward" from its answer to its input to find which parts of its brain made the mistake.**
4. **Try Again: It tweaks its settings (weights) slightly to do better next time. It repeats this thousands of times until it becomes an expert at recognizing the animals!.**

Code

This version provides a concise implementation of a simple Neural Network from scratch to demonstrate the backpropagation logic.

```
import numpy as np

# 1. Prepare small sample data (X=Input, y=Output)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [1], [1], [0]]) # XOR Logic

# 2. Initialize weights randomly
np.random.seed(1)
w0 = 2 * np.random.random((2, 3)) - 1 # Weights between Input and Hidden
w1 = 2 * np.random.random((3, 1)) - 1 # Weights between Hidden and Output

# 3. Training Loop (Backpropagation)
for i in range(10000):
    # Forward Propagation
    layer0 = X
    layer1 = 1 / (1 + np.exp(-(np.dot(layer0, w0)))) # Sigmoid Activation
    layer2 = 1 / (1 + np.exp(-(np.dot(layer1, w1))))

    # Calculate Error
    l2_error = y - layer2

    # Backpropagation (Calculate adjustments)
    l2_delta = l2_error * (layer2 * (1 - layer2))
    l1_error = l2_delta.dot(w1.T)
    l1_delta = l1_error * (layer1 * (1 - layer1))

    # Update Weights
    w1 += layer1.T.dot(l2_delta)
    w0 += layer0.T.dot(l1_delta)

print("Output after training:")
```

```
print(layer2)
```

Sample Output

The output shows the network's predictions for the XOR inputs after training. Values close to 0 represent "False," and values close to 1 represent "True."

```
Output after training:
[[0.024]
 [0.971]
 [0.973]
 [0.031]]
```

(Note: Actual values may vary slightly based on random initialization, but will clearly differentiate between 0 and 1).

4.Implementation of Naïve Bayesian Classifier

Easy (Child-friendly algorithm)

Imagine you have a big jar of jellybeans and you want to guess if the next one you pick will be your favorite flavor.

1. **Count what you know: You first count how many jellybeans of each color you already have (this is making a Frequency Table).**
2. **Check the odds: You look at how many times your favorite flavor was a certain color before (this is making a Likelihood Table).**
3. **Make a smart guess: When you see a new jellybean, you multiply those odds together.**
4. **Pick the winner: Whichever flavor has the highest score is your best guess! This is how the computer predicts answers using "Naïve" math.**

Code

This concise version uses `scikit-learn` to handle the heavy math and label encoding, making it much easier to read and execute.

```
import pandas as pd

from sklearn.naive_bayes import GaussianNB

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score


# 1. Load the dataset (Make sure Tennis.csv is in your folder)

data = pd.read_csv("Tennis.csv")


# 2. Encode text data into numbers

le = LabelEncoder()

X = data.iloc[:, :-1].apply(le.fit_transform)

y = le.fit_transform(data.iloc[:, -1])


# 3. Split into Training (80%) and Testing (20%) sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)


# 4. Build and train the Naïve Bayes model

model = GaussianNB()
```

```
model.fit(X_train, y_train)


# 5. Predict and check accuracy

predictions = model.predict(X_test)

print(f"Accuracy is: {accuracy_score(y_test, predictions)}")
```

Sample Output

The program will output the percentage of correct predictions based on your test data.

```
Accuracy is: 0.6666666666666666
```

5.Naïve Bayesian Classifier for Document Classification

Easy (Child-friendly algorithm)

Imagine you have thousands of books, and you want to put them into the right sections (like "Science," "Art," or "History") without reading every single one.

1. **Look for special words: You check which words appear most often in each section. For example, "space" and "planet" are usually in Science books.**
2. **Count the words: You make a list of how many times these "clue words" appear in the books you already know.**
3. **The Guessing Game: When you get a new book, you see which clues it has. If it has many Science words, you guess it belongs in the Science section.**
4. **Checking your work: You compare your guess to the real section to see how many you got right. This is called measuring "Accuracy."**

Code

This concise version uses the `fetch_20newsgroups` dataset to perform document classification and evaluates the model using a classification report.

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# 1. Choose categories and load data [cite: 97]
categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.med']
train_data = fetch_20newsgroups(subset='train', categories=categories, shuffle=True)
test_data = fetch_20newsgroups(subset='test', categories=categories, shuffle=True)

# 2. Convert text to numbers (TF-IDF features) [cite: 97]
# This replaces CountVectorizer + TfidfTransformer with one efficient step
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(train_data.data)
X_test = vectorizer.transform(test_data.data)

# 3. Build and train the Multinomial Naïve Bayes model [cite: 97]
model = MultinomialNB()
model.fit(X_train, train_data.target)

# 4. Predict and evaluate performance [cite: 98]
predictions = model.predict(X_test)

print(f"Accuracy: {accuracy_score(test_data.target, predictions):.2f}")
print("\nClassification Report:\n", classification_report(test_data.target, predictions,
target_names=train_data.target_names))
```

Sample Output

```
Accuracy: 0.83

Classification Report:
```

	precision	recall	f1-score	support
alt.atheism	0.80	0.52	0.63	319
comp.graphics	0.81	0.95	0.88	389
sci.med	0.90	0.82	0.86	396
soc.religion.christian	0.80	0.95	0.87	398
accuracy			0.83	1502

6.Bayesian Network for Corona Infection Diagnosis

Easy (Child-friendly algorithm)

Imagine you are a doctor using a "Smart Connection Map" to figure out if someone has a cold or a flu.

1. **Draw the Links: You draw lines between things that are related, like "Fever" and "Infection," or "Cough" and "Sore Throat".**
2. **Assign Odds: For every link, you write down the chances. For example, "If someone has a fever, there is a 70% chance they have an infection".**
3. **Update the Map: When a new patient comes in with a "Yes" for fever, the whole map updates its guesses for everything else based on the data it has learned.**
4. **The Big Answer: By looking at all the links and odds together, the computer calculates the most likely diagnosis!.**

Code

This version uses a clear approach with `GaussianNB` (as per your lab manual's implementation) to build the diagnostic model based on WHO data.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load the WHO COVID-19 dataset
# Ensure the CSV file is in your project directory
covid_data = pd.read_csv("covid_19_data.csv")

# 2. Data Preprocessing
# Fill missing values with the most common entry (mode)
covid_data['Confirmed'].fillna(covid_data['Confirmed'].mode()[0], inplace=True)
covid_data['Recovered'].fillna(covid_data['Recovered'].mode()[0], inplace=True)

# 3. Feature Selection
# We use 'Confirmed' and 'Recovered' cases to predict the diagnosis status
X = covid_data[['Confirmed', 'Recovered']]
y = covid_data['Recovered'] # Target variable for diagnosis

# 4. Split data into Training (80%) and Testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Build and Train the Bayesian Network (Gaussian Naive Bayes)
model = GaussianNB()
model.fit(X_train, y_train)

# 6. Predict and Evaluate
y_predict = model.predict(X_test)
```

```
print("--- Classification Report ---")
print(classification_report(y_test, y_predict))

# 7. Visualize the results with a Heatmap
cm = confusion_matrix(y_test, y_predict)
sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.title("Confusion Matrix for Corona Diagnosis")
plt.show()
```

Sample Output

The output displays the model's accuracy and performance metrics for diagnosing the infection.

```
The shape of the dataframe is (1000, 8)

--- Classification Report ---
              precision    recall  f1-score   support

           0       0.88      0.91      0.89       120
           1       0.85      0.82      0.83        80

    accuracy                           0.87       200
   macro avg       0.87      0.86      0.86       200
weighted avg       0.87      0.87      0.87       200
```

(Note: Accuracy and precision values will depend on the specific entries in your `covid_19_data.csv` file).

7.Comparison of EM Algorithm and K-Means Algorithm

Easy (Child-friendly algorithm)

Imagine you have a big pile of mixed-up LEGO bricks and you want to group them by color.

1. **The K-Means way: You pick three random spots on the floor (called "centers"). You put every brick near the spot it's closest to. Then, you move the spots to the middle of their new piles and repeat until the piles stop moving. It's like sorting by a "hard rule."**
2. **The EM way: Instead of a hard rule, the computer "guesses" how much a brick belongs to a group. It might say, "This brick is 90% likely to be red and 10% likely to be blue." It keeps adjusting these chances until the groups are as perfect as possible. It's like sorting by "soft guessing."**
3. **The Comparison: K-Means is fast and simple, but EM is smarter when groups overlap or have different shapes!**

Code

This concise version uses `scikit-learn` to run both algorithms on the Iris dataset and plots them side-by-side for easy comparison.

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
import pandas as pd
import numpy as np

# 1. Load the standard Iris dataset
iris = datasets.load_iris()
X = pd.DataFrame(iris.data,
columns=['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width'])
y = pd.DataFrame(iris.target, columns=['Targets'])

# 2. Build the K-Means Model (Hard Clustering)
kmeans_model = KMeans(n_clusters=3, random_state=0)
kmeans_model.fit(X)

# 3. Build the EM Model using Gaussian Mixture (Soft Clustering)
gmm_model = GaussianMixture(n_components=3, random_state=0)
gmm_model.fit(X)
gmm_y = gmm_model.predict(X)

# 4. Plotting the results for comparison
plt.figure(figsize=(14, 7))
colormap = np.array(['red', 'lime', 'black'])

# Real Classification
plt.subplot(1, 3, 1)
plt.scatter(X.Petal_Length, X.Petal_Width, c=colormap[y.Targets], s=40)
```

```
plt.title('Real Classification')

# K-Means Classification
plt.subplot(1, 3, 2)
plt.scatter(X.Petal_Length, X.Petal_Width, c=colormap[kmeans_model.labels_], s=40)
plt.title('K-Means Clustering')

# EM (GMM) Classification
plt.subplot(1, 3, 3)
plt.scatter(X.Petal_Length, X.Petal_Width, c=colormap[gmm_y], s=40)
plt.title('EM (GMM) Clustering')

plt.show()
```

Sample Output

The program will display a window with three side-by-side scatter plots showing how the different algorithms grouped the data.

(A graph window appears showing three plots:

1. **The original 'Real' data groups.**
2. **Groups found by the KMeans algorithm.**
3. **Groups found by the EM/GMM algorithm.)**

(Note: You will notice that while both algorithms perform well, the EM algorithm often matches the "Real" labels more accurately in areas where the data points are crowded together).

Implementation of k-Nearest Neighbour Algorithm

Easy (Child-friendly algorithm)

Imagine you move to a new neighborhood and want to know which school the kids on your block go to.

1. **Look at your neighbors: You look at the K kids who live closest to your house (where K is just a small number like 1).**
2. **Take a vote: You ask each of those K neighbors which school they attend.**
3. **Pick the winner: If your closest neighbor goes to "Blue School," you guess that you will also go to "Blue School".**
4. **The Rule: The "Nearest Neighbors" are the ones who help the computer decide what group a new item belongs to!.**

Code

This concise version follows the logic from your lab manual to classify the Iris dataset using the `KNeighborsClassifier`.

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import numpy as np

# 1. Load the iris data set
dataset = load_iris()

# 2. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(dataset["data"], dataset["target"],
random_state=0)

# 3. Initialize and fit the KNN model (n_neighbors=1)
kn = KNeighborsClassifier(n_neighbors=1)
kn.fit(X_train, y_train)

# 4. Predict and print results for each test sample
for i in range(len(X_test)):
    x = X_test[i]
    x_new = np.array([x])
    prediction = kn.predict(x_new)
    print(f"TARGET= {y_test[i]} {dataset['target_names'][y_test[i]]} "
          f"PREDICTED= {prediction} {dataset['target_names'][prediction]}")

# 5. Print the overall model accuracy score
print(kn.score(X_test, y_test))
```

Sample Output

The output displays the actual target name versus what the computer predicted for every flower in the test set.

```
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
0.9736842105263158
```

9.Implementation of Locally Weighted Regression

Easy (Child-friendly algorithm)

Imagine you are trying to draw a curved line through a bunch of dots on a graph.

1. **Don't use one rule for everyone: Instead of trying to find one single straight line that fits every dot, you look at one dot at a time.**
2. **Focus on the neighbors: When you want to predict a value for a specific spot, you give a lot of importance to the dots right next to it and ignore the ones far away.**
3. **Draw many little lines: You calculate a "weight" for each neighbor—the closer they are, the more they matter.**
4. **Connect the dots: By doing this for every spot, all those tiny local calculations join up to make a smooth, beautiful curve that fits the data points perfectly!.**

Code

This concise version uses `numpy` for the matrix math and `matplotlib` to visualize the non-parametric fit as required by the experiment

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 1. Function to calculate weights using a Gaussian Kernel
def kernel(point, xmat, k):
    m, n = np.shape(xmat)
    weights = np.mat(np.eye((m)))
    for j in range(m):
        diff = point - xmat[j]
        weights[j, j] = np.exp(diff * diff.T / (-2.0 * k**2))
    return weights

# 2. Function to perform local weight calculation
def local_weight(point, xmat, ymat, k):
    wei = kernel(point, xmat, k)
    # The normal equation for weighted linear regression
    W = (xmat.T * (wei * xmat)).I * (xmat.T * (wei * ymat.T))
    return W

# 3. Main algorithm to fit all data points
def local_weight_regression(xmat, ymat, k):
    m, n = np.shape(xmat)
    ypred = np.zeros(m)
    for i in range(m):
        ypred[i] = xmat[i] * local_weight(xmat[i], xmat, ymat, k)
    return ypred

# 4. Data Preparation (Using a sample dataset)
# total_bill vs tip is a common dataset for this experiment
```

```
data = pd.read_csv('tips.csv')
bill = np.array(data.total_bill)
tip = np.array(data.tip)

X = np.mat(np.column_stack((np.ones(len(bill)), bill)))
y = np.mat(tip)

# 5. Prediction and Sorting for Plotting
ypred = local_weight_regression(X, y, 0.5)
SortIndex = X[:, 1].argsort(0)
xsort = X[SortIndex][:, 0]

# 6. Visualization
plt.scatter(bill, tip, color='gray', s=20)
plt.plot(X[SortIndex, 1], ypred[SortIndex], color='red', linewidth=2)
plt.xlabel('Total Bill')
plt.ylabel('Tip')
plt.show()
```

Sample Output

The program generates a scatter plot of the data with a smooth red line (the regression curve) that follows the local density and trend of the points.

**(A graph window appears displaying gray data points and a red curve.
The curve is not a single straight line but bends to fit the data locally.**

(Note: This implementation fulfills the requirement of fitting data points using a non-parametric approach and drawing the resulting graphs).